



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр 806

Группа М8О-214СВ-24 Направление подготовки 02.03.02 «Фундаментальная информатика и информационные технологии»

Магистерская программа Информатика

Квалификация: магистр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА МАГИСТРА

На тему: Разработка и исследование пулов потоков с механизмом work-stealing на языке C++

Автор ВКРМ: _____ Спиридонов Кирилл Анатольевич (_____)

Научный руководитель: _____ (_____)

Консультант: _____ (_____)

Консультант: _____ (_____)

Рецензент: _____ (_____)

К защите допустить

Заведующий кафедрой № 806 _____ Крылов Сергей Сергеевич (_____)

____ мая 2026 года

Москва 2026

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	3
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	4
1 Анализ предметной области и обоснование темы исследования . . .	5
1.1 Тенденции развития многоядерных и многопроцессорных архитектур	5
1.2 Проблемы наивного подхода к управлению потоками	6
1.3 Накладные расходы на синхронизацию и балансировку нагрузки как нерешённая практическая проблема	8
1.4 Область применения: высоконагруженные серверы, игровые движки, системы реального времени и высокопроизводительные вычисления	10
2 Обзор существующих подходов и решений	13
2.1 Классические модели управления потоками	13
2.2 Механизм work-stealing: история возникновения и теоретические основы	14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе магистра применяют следующие термины с соответствующими определениями:

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе магистра применяют следующие сокращения и обозначения:

1 Анализ предметной области и обоснование темы исследования

1.1 Тенденции развития многоядерных и многопроцессорных архитектур

На протяжении нескольких десятилетий основным способом повышения производительности процессоров являлось увеличение тактовой частоты. Данная стратегия позволяла разработчикам программного обеспечения практически бесплатно получать прирост производительности при переходе на новое поколение оборудования. Однако в начале 2000-х годов этот подход столкнулся с фундаментальными физическими ограничениями: дальнейший рост тактовой частоты приводил к недопустимому увеличению тепловыделения и энергопотребления [1].

Ответом индустрии на данное ограничение стал переход к многоядерным архитектурам. Вместо наращивания производительности одного ядра производители начали размещать на одном кристалле несколько вычислительных ядер, каждое из которых работает на относительно невысокой тактовой частоте. Согласно данным компании Intel, уже к 2006 году большинство новых потребительских процессоров выпускалось с двумя и более ядрами, а к 2023 году флагманские серверные процессоры содержат сотни вычислительных ядер [2].

Данная тенденция нашла отражение в известном высказывании Херба Саттера, ставшем крылатой фразой в сообществе разработчиков: «Бесплатный обед закончился» [1]. Суть этого утверждения состоит в том, что простое обновление оборудования более не даёт автоматического прироста производительности для однопоточных программ. Вместо этого разработчики вынуждены явно проектировать свои приложения с расчётом на параллельное выполнение, чтобы задействовать все доступные вычислительные ресурсы.

Параллельно с развитием многоядерных процессоров активно развивались и многопроцессорные системы, в которых несколько физических процессоров объединяются в единую вычислительную платформу. Такие системы получили широкое распространение в области высокопроизводительных вычислений (HPC), центрах обработки данных и серверных инфраструктурах крупных интернет-компаний. Архитектура NUMA (Non-Uniform Memory Access), характерная для многопроцессорных

систем, вносит дополнительную сложность в разработку параллельных программ: время доступа к памяти зависит от того, на каком узле она физически расположена [3].

Таким образом, к середине 2010-х годов параллелизм превратился из узкоспециализированной темы, интересной лишь разработчикам суперкомпьютеров, в повседневную реальность для широкого круга инженеров-программистов. Веб-серверы, игровые движки, системы машинного обучения, базы данных и пользовательские приложения — все они вынуждены эффективно использовать параллелизм для достижения приемлемой производительности [4].

Рост числа ядер порождает принципиально новые требования к механизмам управления параллельным выполнением задач. Простые решения, приемлемые при двух-четырёх ядрах, демонстрируют неудовлетворительное масштабирование при 32, 64 и более ядрах. Именно поэтому исследование эффективных механизмов управления потоками и распределения задач между ними приобретает особую актуальность в современных условиях.

1.2 Проблемы наивного подхода к управлению потоками

Наиболее очевидным и простым подходом к параллельному выполнению задач является создание нового потока для каждой поступающей задачи и его уничтожение по завершении работы. Данный подход интуитивно понятен, прост в реализации и нередко используется в учебных примерах и прототипах. Однако при применении в производственных системах он обнаруживает ряд серьёзных недостатков.

Высокие накладные расходы на создание и уничтожение потоков. Создание потока операционной системой является дорогостоящей операцией. В операционной системе Linux создание потока через вызов `clone()` занимает порядка 5–15 мкс [5]. В операционной системе Windows аналогичная операция через `CreateThread()` имеет сопоставимую стоимость. При этом каждый поток требует выделения стека (как правило, от 64 КБ до 8 МБ по умолчанию), инициализации структур данных ядра и регистрации в планировщике операционной системы. Для высоконагруженных систем, обрабатывающих тысячи и десятки тысяч задач в секунду, совокупные накладные расходы на управление жизненным циклом

потоков могут составлять значительную долю общего времени выполнения [6].

Неконтролируемый рост числа потоков. В условиях пиковой нагрузки наивный подход приводит к лавинообразному росту числа одновременно существующих потоков. Поскольку каждый поток потребляет память (прежде всего под стек) и ресурсы ядра операционной системы, бесконтрольное создание потоков способно привести к исчерпанию памяти или достижению системного лимита на количество потоков. В Linux данный лимит определяется параметром ядра `/proc/sys/kernel/threads-max` и по умолчанию составляет порядка нескольких тысяч потоков. Превышение этого лимита влечёт отказ в создании новых потоков и, как следствие, отказ в обслуживании запросов [7].

Избыточное переключение контекста. Планировщик операционной системы распределяет процессорное время между всеми активными потоками посредством вытесняющей многозадачности. Переключение контекста — сохранение состояния текущего потока и восстановление состояния следующего — является затратной операцией, требующей нескольких микросекунд и приводящей к инвалидации кэшей процессора. Когда число активных потоков значительно превышает количество ядер, система тратит существенную долю времени на переключение контекстов вместо полезной работы [8]. Данный эффект получил название *thrashing* и является одной из ключевых проблем наивного подхода.

Отсутствие контроля над использованием ресурсов. Наивный подход не предоставляет механизмов ограничения количества одновременно выполняемых задач. В результате в периоды пиковой нагрузки система оказывается перегруженной, что негативно сказывается на латентности обработки всех запросов, включая приоритетные. Концепция *backpressure* — механизма обратного давления, при котором производитель задач замедляется при переполнении системы — принципиально несовместима с наивным подходом [9].

Неэффективное использование кэша процессора. Потоки, созданные для выполнения коротких задач, как правило, не имеют устойчивой привязки к конкретным ядрам процессора. Планировщик ОС может перемещать поток между ядрами при каждом пробуждении, что приводит к систематическим промахам кэша и снижению эффективности работы с памятью. В

современных процессорах разница в латентности между кэшем L1 и основной памятью достигает 200–300 раз, поэтому влияние данного фактора на производительность весьма существенно [10].

Перечисленные недостатки наивного подхода делают его непригодным для применения в производственных системах с высокими требованиями к производительности и предсказуемости. Это обстоятельство обусловило широкое распространение альтернативной концепции — пула потоков, при которой фиксированный набор заранее созданных потоков многократно используется для выполнения поступающих задач. Однако и в рамках концепции пула потоков остаётся ряд нерешённых проблем, связанных с балансировкой нагрузки между потоками, что будет рассмотрено в последующих разделах.

1.3 Накладные расходы на синхронизацию и балансировку нагрузки как нерешённая практическая проблема

Переход к многопоточному программированию неизбежно порождает необходимость координации доступа нескольких потоков к разделяемым ресурсам. Данная координация осуществляется посредством механизмов синхронизации, которые, при всей своей необходимости, сами по себе являются источником существенных накладных расходов и одной из наиболее трудноустраняемых практических проблем в области параллельного программирования [4].

Наиболее распространённым примитивом синхронизации является мьютекс (mutual exclusion lock). Операция захвата свободного мьютекса в Linux на архитектуре x86-64 посредством `pthread_mutex_lock()` занимает порядка 20–40 нс в отсутствие конкуренции. Однако при наличии конкуренции между потоками стоимость возрастает на порядки: поток, не сумевший захватить мьютекс, переводится в состояние ожидания, что влечёт системный вызов, переключение контекста и последующее пробуждение — совокупная стоимость данной цепочки может составлять 1–10 мкс [5]. В высоконагруженных системах, где потоки обращаются к разделяемым ресурсам тысячи раз в секунду, эти задержки накапливаются и становятся определяющим фактором производительности.

Проблема ложного разделения (false sharing). Кэш-память процессора работает с данными не побайтово, а блоками фиксированного размера —

кэш-линиями, типичный размер которых составляет 64 байта. Если два потока, выполняющихся на разных ядрах, обращаются к различным переменным, но эти переменные расположены в одной кэш-линии, каждое обновление одной переменной инвалидирует кэш-линию на другом ядре. В результате возникают непроизводительные синхронизации кэша между ядрами, хотя с логической точки зрения потоки работают с независимыми данными [10]. Данное явление, называемое ложным разделением (false sharing), способно снизить производительность параллельной программы до уровня, сопоставимого с однопоточной реализацией или даже хуже неё.

Проблема АВА и сложность неблокирующих алгоритмов. В попытке избежать накладных расходов на блокирующую синхронизацию разработчики прибегают к неблокирующим (lock-free) алгоритмам, основанным на атомарных операциях процессора, в частности на инструкции `compare-and-swap` (CAS). Однако разработка корректных неблокирующих алгоритмов представляет значительную сложность. Классическим примером является проблема АВА: поток читает значение А, другой поток изменяет его на В, а затем обратно на А, после чего первый поток ошибочно считает, что данные не изменились [11]. Устранение подобных тонких ошибок требует глубокой экспертизы и существенно усложняет реализацию.

Эффективное распределение задач между потоками — балансировка нагрузки — является самостоятельной нетривиальной проблемой. В простейшей модели пула потоков с единой очередью задач все потоки конкурируют за одну разделяемую структуру данных. При большом числе потоков и высокой интенсивности поступления задач данная очередь превращается в узкое место: большую часть времени потоки проводят в ожидании доступа к ней, а не в выполнении полезной работы. Данный эффект получил название `contention` (конкуренция за ресурс) и является фундаментальным ограничением архитектуры с единой очередью [12].

Для устранения конкуренции применяются архитектуры с отдельными очередями задач для каждого потока. Однако данный подход порождает обратную проблему: при неравномерном поступлении задач одни потоки оказываются перегружены, тогда как другие простаивают в ожидании работы. Неравномерность нагрузки может быть обусловлена как характером входящих запросов, так и различиями во времени выполнения отдельных задач. Задача оптимальной балансировки нагрузки в реальном времени без

значительных накладных расходов на координацию остаётся актуальной научно-практической проблемой [13].

Ещё одной проблемой, связанной с синхронизацией, является инверсия приоритетов: высокоприоритетный поток вынужден ожидать освобождения ресурса, захваченного низкоприоритетным потоком, который, в свою очередь, вытеснен потоком со средним приоритетом. В результате высокоприоритетная задача блокируется на неопределённый срок. Данная проблема приобрела широкую известность после инцидента с марсоходом Mars Pathfinder в 1997 году, когда она привела к систематическим перезагрузкам бортового компьютера [14]. В современных системах реального времени инверсия приоритетов по-прежнему остаётся серьёзной угрозой корректности работы.

Совокупность перечисленных проблем — стоимость синхронизации, ложное разделение, сложность неблокирующих алгоритмов, конкуренция за очередь задач, неравномерность нагрузки и инверсия приоритетов — свидетельствует о том, что построение эффективного механизма управления параллельным выполнением задач является нетривиальной инженерной и исследовательской задачей, не имеющей универсального решения в рамках существующих подходов.

1.4 Область применения: высоконагруженные серверы, игровые движки, системы реального времени и высокопроизводительные вычисления

Проблемы эффективного параллельного выполнения задач актуальны для широкого спектра прикладных областей. Ниже рассматриваются наиболее значимые из них с точки зрения требований к механизмам управления потоками.

Современные веб-серверы и серверы приложений обрабатывают сотни тысяч и миллионы запросов в секунду. Такие компании, как Google, Facebook, Amazon и Яндекс, оперируют инфраструктурой, в которой каждый сервер должен поддерживать тысячи одновременных соединений при минимальной латентности отклика [15]. В данном контексте эффективный пул потоков является критически важным компонентом: он определяет пропускную способность сервера и характер деградации под нагрузкой. Неэффективная реализация приводит к резкому росту латентности при приближении к пиковой нагрузке, что непосредственно влияет на пользовательский опыт и

бизнес-показатели.

Характерной особенностью серверных нагрузок является их неоднородность: запросы существенно различаются по сложности и времени обработки, а интенсивность поступления запросов меняется во времени. Это делает статическое распределение задач между потоками неэффективным и требует динамической балансировки нагрузки [16].

Системы реального времени. В системах жёсткого и мягкого реального времени — бортовых авиационных системах, промышленных контроллерах, системах управления транспортными средствами — требования к предсказуемости временных характеристик особенно высоки. Здесь важна не только средняя производительность, но и гарантия того, что задача будет выполнена в пределах установленного временного дедлайна в наихудшем случае (Worst-Case Execution Time, WCET) [17].

Стандартные реализации пулов потоков общего назначения, как правило, не дают подобных гарантий: время ожидания задачи в очереди может варьироваться в широких пределах в зависимости от текущей загрузки системы. Разработка механизмов управления потоками с детерминированными временными характеристиками остаётся открытой исследовательской задачей [18].

Высокопроизводительные вычисления (HPC). В области HPC параллельные вычисления являются основным инструментом решения задач, недоступных для последовательных алгоритмов: молекулярного моделирования, климатических симуляций, задач вычислительной гидродинамики и обучения нейронных сетей [19]. Здесь на первый план выходит эффективность использования вычислительных ресурсов: простой даже небольшой части ядер суперкомпьютера обходится весьма дорого как в денежном, так и во временном отношении.

В HPC-приложениях широко применяется модель разделения задач «разделяй и властвуй» (divide and conquer): крупная задача рекурсивно разбивается на подзадачи вплоть до достижения базового случая. Данная модель естественным образом порождает древовидную структуру зависимостей, в которой динамически создаётся большое количество мелких задач. Именно для данного сценария механизм work-stealing демонстрирует наилучшие теоретические и практические результаты, что будет подробно рассмотрено в следующем разделе [12].

Таким образом, перечисленные области применения объединяет общая потребность в эффективном, масштабируемом и предсказуемом механизме параллельного выполнения задач. Различия в требованиях — максимальная пропускная способность для серверных систем, предсказуемость латентности для игровых движков, детерминизм для систем реального времени и эффективность использования ресурсов для НРС — определяют многообразие подходов к реализации пулов потоков и обуславливают актуальность их исследования.

2 Обзор существующих подходов и решений

2.1 Классические модели управления потоками

В процессе развития параллельного программирования сложилось несколько устойчивых моделей управления потоками, каждая из которых представляет собой определённый компромисс между простотой реализации, накладными расходами и эффективностью использования ресурсов.

Исторически первой и наиболее простой является модель «поток на задачу» (thread-per-task), при которой для каждой входящей задачи создаётся отдельный поток, уничтожаемый по завершении работы. Достоинством данной модели является концептуальная простота: разработчику не требуется явно управлять жизненным циклом потоков или координировать распределение задач. Недостатки этой модели — высокие накладные расходы на создание и уничтожение потоков.

Следующим шагом в развитии стала модель пула потоков с единой глобальной очередью (single global queue thread pool). В данной модели при запуске приложения создаётся фиксированное число потоков, равное, как правило, числу логических ядер процессора. Все потоки разделяют одну очередь задач: производитель помещает задачи в конец очереди, а потоки-потребители извлекают их из начала. Данная модель устраняет накладные расходы на создание и уничтожение потоков, однако порождает новую проблему: разделяемая очередь становится точкой конкуренции. При большом числе потоков операции постановки и извлечения задач требуют синхронизации посредством мьютекса или атомарных операций, что ограничивает масштабируемость системы [20].

Для устранения конкуренции за единую очередь была предложена модель пула потоков с локальными очередями (per-thread queue thread pool). В данной архитектуре каждый поток владеет собственной очередью задач. Новые задачи назначаются потокам по некоторому правилу — например, по принципу Round Robin или путём хеширования идентификатора задачи. Данный подход снижает конкуренцию, однако приводит к статическому распределению задач, которое не адаптируется к реальной загрузке потоков. В результате возможна ситуация, когда одни потоки перегружены, тогда как другие простаивают [4].

Попыткой совместить преимущества обеих моделей стала архитектура с

иерархическими очередями (hierarchical queue). В ней глобальная очередь выступает резервуаром задач, а каждый поток дополнительно располагает локальным буфером для хранения небольшого числа задач. Поток в первую очередь берёт задачи из локального буфера, обращаясь к глобальной очереди лишь при его опустошении. Данный подход снижает частоту синхронизации, но не решает проблему неравномерной загрузки потоков [20].

Отдельного рассмотрения заслуживает модель реакторов (reactor pattern) и её расширение — модель SEDA (Staged Event-Driven Architecture). В модели реактора единственный поток обрабатывает события ввода-вывода и диспетчеризирует задачи, что позволяет избежать накладных расходов на синхронизацию, однако не позволяет эффективно задействовать несколько ядер. Модель SEDA разделяет обработку на несколько последовательных стадий, каждая из которых обслуживается собственным пулом потоков, что обеспечивает лучшую масштабируемость и управляемость нагрузки [16].

Анализ перечисленных моделей позволяет выявить фундаментальное противоречие, характерное для классических подходов: снижение конкуренции за разделяемые структуры данных достигается ценой ухудшения балансировки нагрузки, и наоборот. Данное противоречие является основной мотивацией для разработки механизма work-stealing, который стремится одновременно минимизировать синхронизацию и обеспечить динамическую балансировку нагрузки.

2.2 Механизм work-stealing: история возникновения и теоретические основы

Механизм work-stealing (кража задач) представляет собой алгоритм динамического распределения задач между потоками, при котором незагруженный поток вправе забирать («красть») задачи из очередей других, более загруженных потоков. Данный подход позволяет совместить низкую конкуренцию за локальные очереди с автоматической балансировкой нагрузки, не требующей централизованного координатора.

Истоки механизма work-stealing восходят к исследованиям в области параллельных вычислений, проводившимся в 1980-е годы. Одной из первых систем, реализовавших схожие принципы, стал язык Multilisp, разработанный Робертом Хэлстедом в Массачусетском технологическом институте в 1985 году [21]. В Multilisp параллельные вычисления организовывались

посредством конструкции `rcall`, а распределение задач между процессорами осуществлялось динамически.

Однако подлинной теоретической основой современного work-stealing стала система Cilk, разработанная в MIT в 1990-е годы группой под руководством Чарльза Лейзерсона. В основополагающей работе Блюмофе и Лейзерсона [12] была формализована модель многопоточных вычислений и доказана оптимальность жадного алгоритма work-stealing с точки зрения теоретической эффективности.

В модели Блюмофе и Лейзерсона многопоточное вычисление представляется в виде ориентированного ациклического графа (DAG), вершины которого соответствуют элементарным инструкциям, а рёбра задают отношения зависимости. Для данной модели вводятся два ключевых параметра: работа T_1 — суммарное число инструкций при выполнении на одном процессоре, и *глубина* (критический путь) T_∞ — длина наидлиннейшей цепочки зависимых инструкций.

Авторы доказали, что при выполнении на P процессорах алгоритм work-stealing обеспечивает время выполнения:

$$T_P \leq \frac{T_1}{P} + O(T_\infty) \quad (1)$$

Данная оценка является оптимальной с точностью до константного множителя: первое слагаемое отражает теоретический минимум, обусловленный объёмом работы, второе — неустранимые потери из-за последовательных зависимостей [12].

Помимо оценки времени выполнения, Блюмофе и Лейзерсон доказали, что алгоритм work-stealing является оптимальным по использованию памяти: потребление памяти при выполнении на P процессорах не превышает P -кратного потребления при последовательном выполнении. Данное свойство принципиально важно для практических приложений, поскольку бесконтрольный рост потребления памяти является типичной проблемой наивных реализаций параллельных вычислений.

Ключевым структурным элементом алгоритма work-stealing является двусторонняя очередь (double-ended queue, deque), которой владеет каждый поток. Работа с данной структурой данных подчиняется следующим правилам:

- поток-владелец помещает новые задачи в нижний конец очереди (push) и извлекает задачи для выполнения также из нижнего конца (pop), то есть работает по принципу стека (LIFO);
- поток-«вор», не имеющий собственных задач, извлекает задачи из верхнего конца очереди жертвы (steal), то есть работает по принципу очереди (FIFO).

Такое асимметричное использование deque преследует две цели. Во-первых, доступ владельца к нижнему концу в большинстве случаев не требует синхронизации, поскольку конкуренция возможна лишь в граничной ситуации, когда очередь содержит ровно одну задачу. Во-вторых, вору забирают задачи с верхнего конца, то есть наиболее «старые» задачи, порождённые ранее всего. В рекурсивных алгоритмах типа «разделяй и властвуй» такие задачи, как правило, являются наиболее крупными и предоставляют больше всего работы, что делает кражу максимально эффективной [13].

Важным практическим достижением стала работа Чейза и Лева [22], в которой была предложена реализация deque для work-stealing на основе динамически расширяемого кольцевого буфера. Данная реализация требует синхронизации (посредством инструкции CAS) лишь в случае конкуренции между владельцем и вором за последнюю задачу в очереди, обеспечивая тем самым практически неблокирующее поведение в типичных сценариях.

После публикации теоретических работ механизм work-stealing был реализован в ряде промышленных систем и языков программирования. Библиотека Intel Threading Building Blocks (TBB), выпущенная в 2006 году, стала одной из первых широко используемых реализаций work-stealing для языка C++ [12]. В языке Java механизм work-stealing был включён в стандартную библиотеку в виде класса ForkJoinPool, появившегося в версии Java 7 в 2011 году. В языке C++ стандарт не предоставляет встроенной реализации work-stealing, что обуславливает актуальность разработки собственных эффективных реализаций [20].

Таким образом, механизм work-stealing сочетает в себе строгое теоретическое обоснование и практическую эффективность, что делает его одним из наиболее перспективных подходов к управлению задачами в параллельных системах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Sutter H. The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software // Dr. Dobbs's Journal. — 2005. — Т. 30, № 3. — С. 202—210.
2. Intel Corporation. Intel® Xeon® Scalable Processors. — 2023. — URL: <https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable.html> (дата обращения 01.11.2025).
3. Lameter C. NUMA (Non-Uniform Memory Access): An Overview // ACM Queue. — 2013. — Т. 11, № 7. — С. 40—51. — DOI: 10.1145/2508834.2513149.
4. The Art of Multiprocessor Programming / М. Herlihy [и др.]. — 2-е изд. — Cambridge, MA : Morgan Kaufmann, 2020.
5. Drepper U., Molnar I. The Native POSIX Thread Library for Linux. — Red Hat Inc., 2005.
6. Butenhof D. R. Programming with POSIX Threads. — Boston, MA : Addison-Wesley, 1997.
7. Love R. Linux Kernel Development. — 3-е изд. — Boston, MA : Addison-Wesley, 2010.
8. Tanenbaum A. S., Bos H. Modern Operating Systems. — 4-е изд. — Hoboken, NJ : Pearson, 2014.
9. Reactive Streams Working Group. Reactive Streams Specification. — 2014. — URL: <https://www.reactive-streams.org> (дата обращения 07.12.2025).
10. Drepper U. What Every Programmer Should Know About Memory. — Red Hat Inc., 2007.
11. Michael M. M., Scott M. L. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms // Proceedings of the 15th Annual ACM Symposium on Principles of Distributed Computing. — 1996. — С. 267—275. — DOI: 10.1145/248052.248106.
12. Blumofe R. D., Leiserson C. E. Scheduling Multithreaded Computations by Work Stealing // Journal of the ACM. — 1999. — Т. 46, № 5. — С. 720—748. — DOI: 10.1145/324133.324234.

13. Acar U. A., Blelloch G. E., Blumofe R. D. The Data Locality of Work Stealing // Proceedings of the 12th Annual ACM Symposium on Parallel Algorithms and Architectures. — 2000. — С. 1—12. — DOI: 10.1145/341800.341801.
14. Reeves G. E. What Really Happened on Mars? // IEEE Spectrum. — 1998. — Т. 35, № 2. — С. 18—25.
15. Wiggins A. The Twelve-Factor App. — 2012. — URL: <https://12factor.net> (дата обращения 15.12.2025).
16. Welsh M., Culler D., Brewer E. SEDA: An Architecture for Well-Conditioned, Scalable Internet Services // ACM SIGOPS Operating Systems Review. — 2001. — Т. 35, № 5. — С. 230—243. — DOI: 10.1145/502059.502057.
17. The Worst-Case Execution-Time Problem — Overview of Methods and Survey of Tools / R. Wilhelm, J. Engblom, A. Ermedahl [и др.] // ACM Transactions on Embedded Computing Systems. — 2008. — Т. 7, № 3. — С. 1—53. — DOI: 10.1145/1347375.1347389.
18. An EDF Scheduling Class for the Linux Kernel / D. Faggioli [и др.] // Proceedings of the 11th Real-Time Linux Workshop. — 2009. — С. 1—10.
19. Dongarra J., Heroux M. A., Luszczek P. High-Performance Conjugate-Gradient Benchmark: A New Metric for Ranking High-Performance Computing Systems // The International Journal of High Performance Computing Applications. — 2016. — Т. 30, № 1. — С. 3—10. — DOI: 10.1177/1094342015593158.
20. Williams A. C++ Concurrency in Action. — 2-е изд. — Shelter Island, NY : Manning Publications, 2019.
21. Halstead R. H. Multilisp: A Language for Concurrent Symbolic Computation // ACM Transactions on Programming Languages and Systems. — 1985. — Т. 7, № 4. — С. 501—538. — DOI: 10.1145/4472.4478.
22. Chase D., Lev Y. Dynamic Circular Work-Stealing Deque // Proceedings of the 17th Annual ACM Symposium on Parallelism in Algorithms and Architectures. — 2005. — С. 21—28. — DOI: 10.1145/1073970.1073974.